

Rodrigo Fernandez

Senior React.js / Python (Django/FastAPI) Full Stack Engineer

Katy, Texas | +1 (407) 801-1287 | www.linkedin.com/in/rodrigobermejo-fernández-9a5a0664 | rodrigobfdev@gmail.com

Summary

Senior **React.js 16–18 / Python 3.8–3.12 Full Stack Engineer** with 12+ years building secure, high-traffic platforms across **cloud infrastructure, healthcare SaaS, eCommerce, digital marketing, and enterprise productivity** industries, specializing in accessible React interfaces, Django/FastAPI APIs, PostgreSQL data models, AWS deployments, CI/CD automation, and production debugging, with strong experience migrating legacy UIs and Python services to modern versions, reducing latency, improving release quality, and translating ambiguous product needs into scalable features that supported safer data workflows and cleaner engineering collaboration across distributed agile teams.

Skills

- **Frontend: React.js 16–18, TypeScript 3.x–5.x**, JavaScript ES6+, Redux, Redux Toolkit, React Query, React Router, Formik, React Hook Form, Material UI, Tailwind CSS, HTML5, CSS3, SASS, Webpack, Vite, Jest, React Testing Library, Cypress
- **Backend: Python 3.8–3.12, Django 3.x–4.x, Django REST Framework, FastAPI 0.8x–0.11x**, Flask, Celery, REST APIs, GraphQL, WebSockets, JWT, OAuth2, RBAC, API versioning, background jobs, async processing
- **Database: PostgreSQL 11–15**, MySQL, MongoDB, Redis, SQLAlchemy, Django ORM, Alembic, query optimization, indexing, migrations, stored procedures, data validation
- **Cloud & DevOps: AWS EC2, S3, RDS, Lambda, ECS, CloudWatch**, Docker, GitHub Actions, GitLab CI/CD, Nginx, Linux, Terraform basics, CI/CD pipelines, release automation, monitoring, logging
- **Engineering Practices:** Agile/Scrum, system design, microservices, monolith modernization, accessibility, performance tuning, code reviews, secure coding, unit testing, integration testing, production incident resolution

Experience

Lightedge — Senior Software Engineer *(Apr 2020 – Mar 2026)*

- Led development of **React.js 17–18** dashboards and **Python 3.9–3.12 FastAPI** services for cloud infrastructure customers, improving workflow visibility and reducing support escalations by **28%**.
- Migrated several legacy class-based React modules into functional components with hooks, cleaner state boundaries, and reusable UI patterns that reduced duplicate frontend code by **32%**.
- Designed secure **Python FastAPI** endpoints for billing, service provisioning, and customer operations, using PostgreSQL transactions to prevent partial updates during failed workflow execution.
- Resolved a recurring production issue where stale React Query cache returned outdated service status, adding cache invalidation rules and websocket refresh events to restore accurate real-time views.
- Improved API performance by profiling slow Django ORM queries, adding PostgreSQL indexes, and reducing average dashboard load time by **41%** during peak customer usage.
- Implemented role-based access control with **JWT, OAuth2, and permission guards**, preventing unauthorized admin actions across sensitive infrastructure management screens.

- Built reusable React form components with schema validation, error boundaries, and typed API contracts, helping product teams release complex customer onboarding flows faster.
- Modernized deployment workflows with Docker and GitHub Actions, adding automated test gates that reduced failed production releases by **35%** across React and Python services.
- Fixed a memory leak in a long-running Python worker by tracing unclosed database sessions, refactoring Celery task cleanup, and adding CloudWatch alerts for worker health.
- Collaborated with DevOps teams to improve AWS observability using structured logs, CloudWatch metrics, and alert thresholds that made service incidents easier to triage.
- Introduced frontend accessibility improvements across modals, tables, and keyboard navigation, helping internal platforms meet stronger usability expectations for enterprise clients.
- Mentored engineers through code reviews, architecture discussions, and debugging sessions, encouraging flexible use of **React**, **Python**, and SQL based on feature complexity.

NIX United — Software Engineer *(Oct 2016 – Mar 2020)*

- Built healthcare and SaaS applications using **React.js 16–17**, **Python 3.7–3.9**, Django REST Framework, PostgreSQL, and Redis for secure patient, provider, and business workflows.
- Implemented dynamic React data grids, filtering, and export features for operations teams, reducing manual reporting work by **30%** through cleaner self-service interfaces.
- Migrated older jQuery-heavy screens into React components, preserving business behavior while improving maintainability, testing coverage, and future feature delivery speed.
- Developed Django REST APIs for scheduling, document handling, and notification modules, applying serializer validation to reduce malformed production requests by **24%**.
- Solved a difficult timezone bug in appointment workflows by standardizing UTC storage, browser-local rendering, and backend validation across Python and React layers.
- Added background processing with Celery and Redis for email notifications and report generation, preventing long-running tasks from blocking customer-facing API requests.
- Improved authentication reliability by tightening token refresh behavior, frontend route guards, and backend permission checks for multi-role healthcare application users.
- Refactored PostgreSQL queries and Django ORM relationships that caused duplicate records, adding constraints and migrations to protect data integrity during updates.
- Supported migration from older Python dependencies to newer stable versions, resolving package conflicts and improving deployment repeatability across development environments.
- Created Jest and React Testing Library coverage for high-risk form workflows, helping QA catch regressions before release and improving release confidence by **27%**.
- Worked closely with product owners and QA teams to clarify ambiguous feature requirements, turning rough business rules into stable React screens and Python services.

Webiz Digital — Software Developer *(Jan 2014 – Sep 2016)*

- Developed eCommerce and digital marketing platforms with **React.js 15–16**, **Python 3.5–3.7**, Django, PostgreSQL, and REST APIs for customer-facing campaign systems.
- Built responsive React landing pages, checkout enhancements, and admin workflows that improved campaign conversion tracking and reduced frontend maintenance effort.
- Implemented Python/Django APIs for product catalogs, promotion rules, and customer activity tracking, giving marketing teams cleaner access to campaign performance data.
- Resolved a cart pricing bug caused by inconsistent frontend rounding and backend decimal handling, standardizing calculations in Python and improving checkout accuracy.
- Migrated selected legacy templates into reusable React components, allowing designers and developers to share UI patterns across multiple campaign microsites.

- Optimized PostgreSQL queries for product search and campaign dashboards, reducing slow admin page response time by **38%** after indexing and query cleanup.
- Added error handling and retry logic for third-party API integrations, preventing failed payment and email provider calls from silently breaking customer workflows.
- Created reusable form validation utilities for React and Django, reducing repeated validation bugs across lead capture, checkout, and account management screens.
- Improved deployment stability by documenting release steps, adding smoke tests, and coordinating rollback plans for high-traffic promotional launches.
- Collaborated flexibly across frontend, backend, and QA tasks, using **React** for customer interaction issues and **Python** for business logic reliability.

Aurio — Junior Software Developer *(Oct 2012 – Dec 2013)*

- Supported enterprise productivity tools using **JavaScript**, **early React patterns**, **Python 2.7–3.4**, Django, MySQL, and REST integrations for internal business users.
- Built and maintained UI components for task tracking, document review, and internal reporting screens, improving usability for non-technical operations teams.
- Assisted senior engineers with Python backend fixes, including validation errors, missing edge-case handling, and inconsistent API responses from older services.
- Helped migrate selected Python scripts from **Python 2.7** toward **Python 3.x**, updating syntax, dependencies, and test cases to reduce long-term maintenance risk.
- Fixed a recurring form submission issue caused by duplicate browser events, adding frontend guards and backend idempotency checks to prevent repeated records.
- Improved MySQL query performance for internal reports by reviewing joins, adding indexes, and reducing unnecessary data loading on frequently used pages.
- Created small automation scripts in Python for data cleanup, CSV imports, and operational reporting, saving support teams several hours of manual work each week.
- Participated in code reviews and QA verification, learning to balance fast delivery with readable code, safer database changes, and practical test coverage.
- Worked across frontend and backend tasks as needed, building strong flexibility with **JavaScript**, **Python**, SQL, debugging, and production support workflows.

Microsoft — Software Developer Intern *(Jun 2012 – Sep 2012)*

- Supported internal engineering tools using **C#**, **.NET Framework 4.0–4.5**, **ASP.NET MVC 3–4**, JavaScript, jQuery, and SQL Server, gaining practical experience in enterprise software quality and structured development workflows.
- Assisted senior engineers with debugging UI defects, validation issues, and data-handling problems, documenting root causes clearly so fixes could be reviewed and released with lower regression risk.
- Built small internal web modules for tracking records, reviewing status changes, and improving team visibility, applying clean HTML, CSS, JavaScript, and backend integration patterns.
- Resolved early-stage bugs related to inconsistent form validation and SQL query results by reproducing issues locally, checking logs, and working with mentors to apply safer input-handling rules.

Education

Texas State University

Bachelor's Degree in Computer Science *(Sep 2008 – May 2012)*